Decision Trees — Practice Problems

Instructor: Dr. Akash Gupta

Table of contents

Part 1: Questions **2**

Part 2: Answers and Explanations **23**

Part 1: Questions

Question 1

What is a decision tree, in plain English?

a) A predictive model that asks a sequence of yes/no questions about the features and routes each row to a "leaf" that produces a prediction
b) A model that predicts the mean of the target on every input
c) A linear model with regularization
d) A neural network with one hidden layer
e) A clustering algorithm

☐ *Jump to solution*

Question 2

Which task can a decision tree solve?

a) Only classification
b) Only regression
c) Both classification and regression — `DecisionTreeClassifier` predicts the majority class at each leaf; `DecisionTreeRegressor` predicts the mean of the training targets at each leaf
d) Only unsupervised problems
e) Only multiclass problems with three or more classes

☐ *Jump to solution*

Question 3

A classification decision tree has reached a leaf containing 80 training customers: 60 churned and 20 stayed. What does the tree predict for a NEW customer that lands at this leaf, with default settings?

a) NaN until the tree is retrained
b) Stay — the minority class
c) A 50/50 random guess
d) A regression value (the average customer age)
e) Churn — the majority class in the leaf — and a predicted probability of about 60 / 80 = 0.75

☐ *Jump to solution*

Question 4

A REGRESSION decision tree leaf contains five training homes with sale prices $300K, $320K, $330K, $340K, $360K. What does the tree predict for a NEW home that lands at this leaf?

a) The minimum: $300K
b) The maximum: $360K
c) The mean: $330K
d) The median: $330K
e) A random sample from those five values

☐ *Jump to solution*

Question 5

When does a decision tree typically OUTPERFORM a plain linear / logistic regression?

a) When the relationship is strictly linear
b) When the features are perfectly uncorrelated
c) When the target is exactly normally distributed
d) When the dataset has fewer than 5 rows
e) When the relationship between features and target is non-linear, has interactions, or has step-changes — trees handle these natively without manually engineered transformations

☐ *Jump to solution*

Question 6

What does a single PATH from the root to a leaf in a fitted decision tree represent?

a) The model's loss curve
b) The training set's standard deviation
c) The model's intercept
d) A set of rules ("feature_a > 5 AND feature_b $\leq$ 10 AND …") that defines the conditions a row must meet to land at that leaf — and the prediction the tree makes for any row matching all those rules
e) Random noise

☐ *Jump to solution*

Question 7

Which formula correctly defines **Gini impurity** for a binary classification node where p is the proportion of class 1?

a) $1 - p^2$
b) $1 - (p^2 + (1-p)^2)$
c) $1 - p$
d) $-p \log p$
e) $p(1-p)^2$

☐ *Jump to solution*

Question 8

A node contains 100 training customers, ALL of whom churned (a "pure" leaf). What is its Gini impurity?

a) 0.50
b) 0.25
c) 0.00
d) 0.75
e) 1.00

☐ *Jump to solution*

Question 9

A binary-classification node has 100 customers split 50 / 50 between the two classes. What is its Gini impurity?

a) 0.00
b) 0.25
c) 0.75
d) 0.50
e) 1.00

☐ *Jump to solution*

Question 10

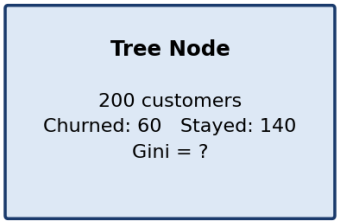
A node contains 100 customers: 25 churned and 75 stayed. What is its Gini impurity?

a) 0.250
b) 0.500
c) 0.375
d) 0.625
e) 0.750

☐ *Jump to solution*

Question 11

A decision-tree node contains 200 customers — 60 churned and 140 stayed.



What is its Gini impurity?

a) 0.30
b) 0.58
c) 0.42
d) 0.70
e) 0.50

☐ *Jump to solution*

Question 12

What is the MAXIMUM possible Gini impurity for a binary classification leaf?

a) 1.0
b) 0.5 — reached when the leaf is split exactly 50 / 50 between the two classes
c) 2.0
d) 0.25
e) There is no upper bound

☐ *Jump to solution*

Question 13

A team can choose `criterion="gini"` or `criterion="entropy"` for `DecisionTreeClassifier`. Which is the MOST accurate practical guidance?

a) Both measure node impurity and almost always produce very similar trees in practice; Gini is slightly faster (no logarithm) and is sklearn's default
b) Entropy is mathematically guaranteed to produce better trees
c) Gini only works on binary problems; entropy is required for multiclass
d) Gini is for regression, entropy for classification
e) Entropy is deprecated in scikit-learn

☐ *Jump to solution*

Question 14

How does a REGRESSION decision tree decide where to split (its splitting criterion)?

a) Maximize Gini impurity
b) Pick splits that REDUCE the variance of the target within each child node — equivalently, minimize the weighted MSE across the children
c) Maximize the cross-entropy at the root
d) Pick the median of the target column
e) Pick splits that maximize the depth

☐ *Jump to solution*

Question 15

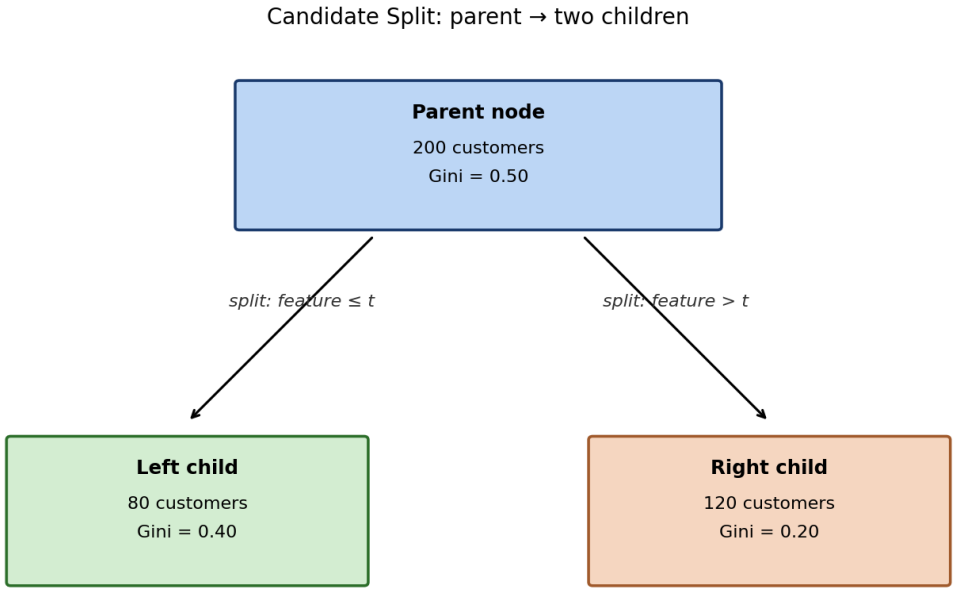
What does **information gain** measure when a tree evaluates a candidate split?

a) How much PURER the children are than the parent — specifically, parent impurity minus the weighted average impurity of the children. The tree picks the split that maximizes this number
b) The training accuracy if the tree stopped at that split
c) The depth added to the tree by that split
d) The number of features in the dataset
e) The total number of leaves the tree will eventually have

☐ *Jump to solution*

Question 16

A candidate split on a churn dataset takes a parent node and produces two children, as shown below.



- Parent node: Gini = 0.50, 200 customers
- Left child: 80 customers, Gini = 0.40
- Right child: 120 customers, Gini = 0.20

Compute the information gain of this split.

a) 0.22
b) 0.18
c) 0.10
d) 0.30
e) 0.50

☐ *Jump to solution*

Question 17

Why does a tree always pick the split with the HIGHEST information gain at each node (rather than the lowest)?

a) Lower information gain produces faster trees
b) Higher information gain means deeper trees, which is always better
c) Lower information gain would cause the tree to overfit
d) sklearn requires it

e) Higher information gain means the split makes the children PURER than the parent — exactly the direction the tree wants to move toward leaves where prediction is unambiguous

☐ *Jump to solution*

Question 18

What does the hyperparameter `max_depth=5` control in `DecisionTreeClassifier`?

a) The maximum number of features the tree is allowed to use
b) The maximum number of training rows allowed in any leaf
c) The minimum accuracy the tree must reach
d) The maximum number of LEVELS from the root to any leaf — a hard pre-pruning cap on tree complexity
e) The number of trees in the ensemble

☐ *Jump to solution*

Question 19

What does `min_samples_leaf=50` do?

a) Sets the minimum number of training samples that must end up in any leaf — splits that would create a leaf smaller than this are not allowed
b) Drops 50 samples from training
c) Forces every node to have exactly 50 children
d) Sets the maximum number of leaves
e) Forces the model to use 50 features

☐ *Jump to solution*

Question 20

What does `min_samples_split=20` do?

a) Sets the maximum number of leaves to 20
b) Drops every node with fewer than 20 samples
c) A node will be considered for further splitting only if it contains at least 20 samples; otherwise it becomes a leaf
d) Forces every leaf to have exactly 20 samples
e) Sets the random seed to 20

☐ *Jump to solution*

Question 21

What does the `ccp_alpha` parameter do in scikit-learn's decision tree?

a) Sets the maximum tree depth directly
b) Controls cost-complexity (post-) pruning — after fully growing the tree, branches whose contribution to predictive accuracy is smaller than the alpha penalty are pruned away. Larger alpha = more aggressive pruning = smaller tree
c) Sets the random seed
d) Controls the number of features per split
e) Selects between Gini and entropy

☐ *Jump to solution*

Question 22

Why do data scientists routinely set `random_state=42` when fitting a tree?

a) sklearn requires it; otherwise the tree refuses to train
b) It seeds the random tie-breaking when multiple splits produce equal impurity reductions, so the same code produces the same tree on every run — essential for reproducibility
c) It improves accuracy
d) 42 is the only legal value
e) It selects the model

☐ *Jump to solution*

Question 23

What does `max_features="sqrt"` do at each split?

a) Sets the maximum tree depth to the square root of the number of features
b) Means "use only one feature per split"
c) Drops the square root of the rows
d) Disables splitting entirely
e) At every split, the tree only considers a RANDOM subset of $\sqrt{p}$ features (where p is the total number of features). Used in Random Forest to make individual trees diverse from each other

☐ *Jump to solution*

Question 24

A telecom team trains a decision tree on a churn dataset that is 5% churners and 95% non-churners. Without any adjustment, the tree predicts "no churn" for nearly every customer. Which sklearn parameter is the standard first remedy?

a) `random_state=42`
b) `max_depth=10000`
c) `criterion="entropy"`
d) `min_samples_split=1`
e) `class_weight="balanced"` — adjusts the loss so minority-class errors count more heavily during fitting, so the tree pays attention to the rare class

☐ *Jump to solution*

Question 25

Which statement correctly distinguishes **pre-pruning** from **post-pruning**?

a) Pre-pruning grows the full tree first, then trims; post-pruning sets hyperparameters
b) Pre-pruning only works for regression
c) They are identical
d) Pre-pruning sets hyperparameters (e.g., `max_depth`, `min_samples_leaf`, `min_samples_split`) BEFORE fitting to stop the tree from growing too complex; post-pruning grows the full tree and then removes branches that do not improve validation performance (typically via `ccp_alpha`)
e) Post-pruning is deprecated

☐ *Jump to solution*

Question 26

Why DOES a fully-grown, unrestricted decision tree often overfit?

a) Without restrictions it keeps splitting until each leaf contains a tiny number (often one) of training points — the tree memorizes the training data perfectly but the splits in late levels are fitting noise rather than signal
b) Its loss function is wrong
c) It does not use enough features
d) Trees can never overfit
e) sklearn refuses to train large trees

☐ *Jump to solution*

Question 27

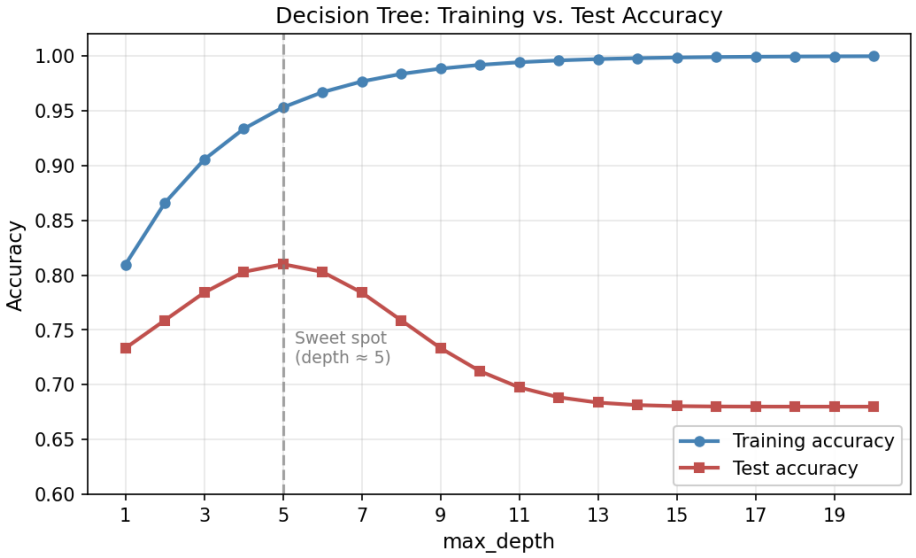
A telecom team trains a decision tree on a churn dataset with no pruning and gets **train accuracy = 0.99** and **test accuracy = 0.68**. What is the BEST first response?

a) The tree is overfitting. Apply pre-pruning (lower `max_depth`, raise `min_samples_leaf`) or post-pruning (raise `ccp_alpha`); use cross-validation to pick the hyperparameters
b) Deploy the model — high training accuracy is excellent
c) Add more features
d) Train longer
e) Switch to KNN

☐ *Jump to solution*

Question 28

A team trains decision trees at many `max_depth` values and plots the result.



What is the BEST takeaway from the plot?

a) Use the deepest tree possible — train accuracy is highest there
b) The plot is unreliable; ignore
c) Decision trees cannot be tuned with depth
d) Train accuracy keeps rising with depth, but test accuracy peaks at a moderate depth and then declines as the tree starts overfitting; pick `max_depth` near the peak
e) Both curves rise indefinitely

☐ *Jump to solution*

Question 29

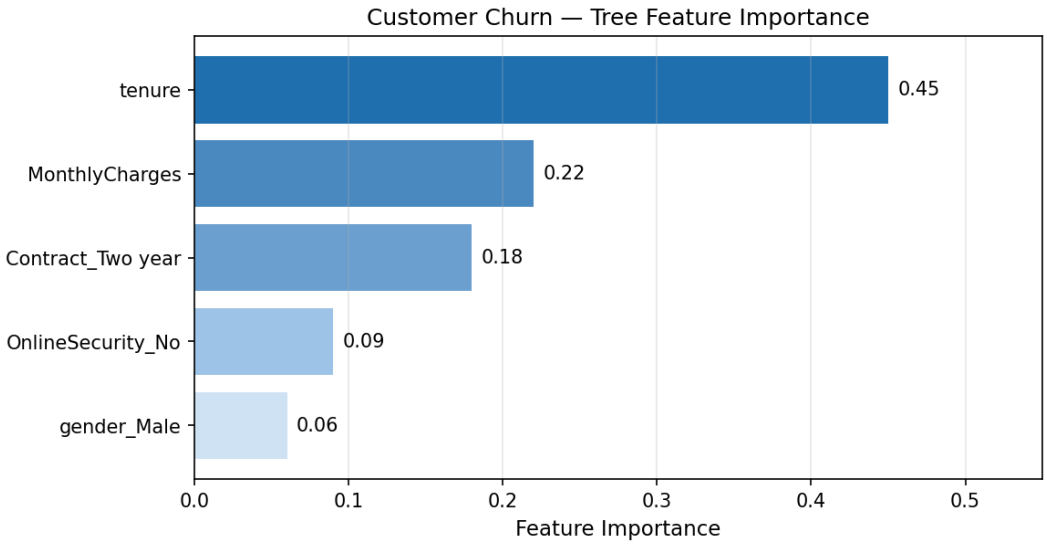
What does the `feature_importances_` attribute of a fitted scikit-learn tree represent?

a) The Pearson correlation of each feature with the target
b) The percentage of customers in each feature category
c) The number of times each feature appears in the data
d) The p-value of each feature
e) The normalized total reduction in impurity (Gini, by default) brought by splits on each feature, summed across the tree — feature importances always sum to 1.0

☐ *Jump to solution*

Question 30

A telecom team plots a decision tree's feature importances for a churn model:



Which interpretation is BEST?

a) The tree relied most heavily on `tenure`, then `MonthlyCharges` and `Contract_Two year`; `gender_Male` contributed almost nothing. Importance reflects model usage, not causation
b) Gender is a strong predictor of churn for this model
c) Customer tenure causes them to churn
d) The chart proves the dataset has exactly five features
e) Tenure should be removed from the model

☐ *Jump to solution*

Question 31

A bank's decision tree assigns the highest feature importance to `customer_id` (a unique numeric identifier). What is going on, and what should the analyst do?

a) Trust the model — `customer_id` is genuinely predictive
b) Impurity-based importance is inflated by high-cardinality features (each unique value can isolate one row, looking very informative on training data but having no generalization). DROP `customer_id` and any other ID-like columns; cross-check with permutation importance
c) Build a lookup table from `customer_id` to default risk
d) Add more ID-like features
e) Add a regularization term to the tree

☐ *Jump to solution*

Question 32

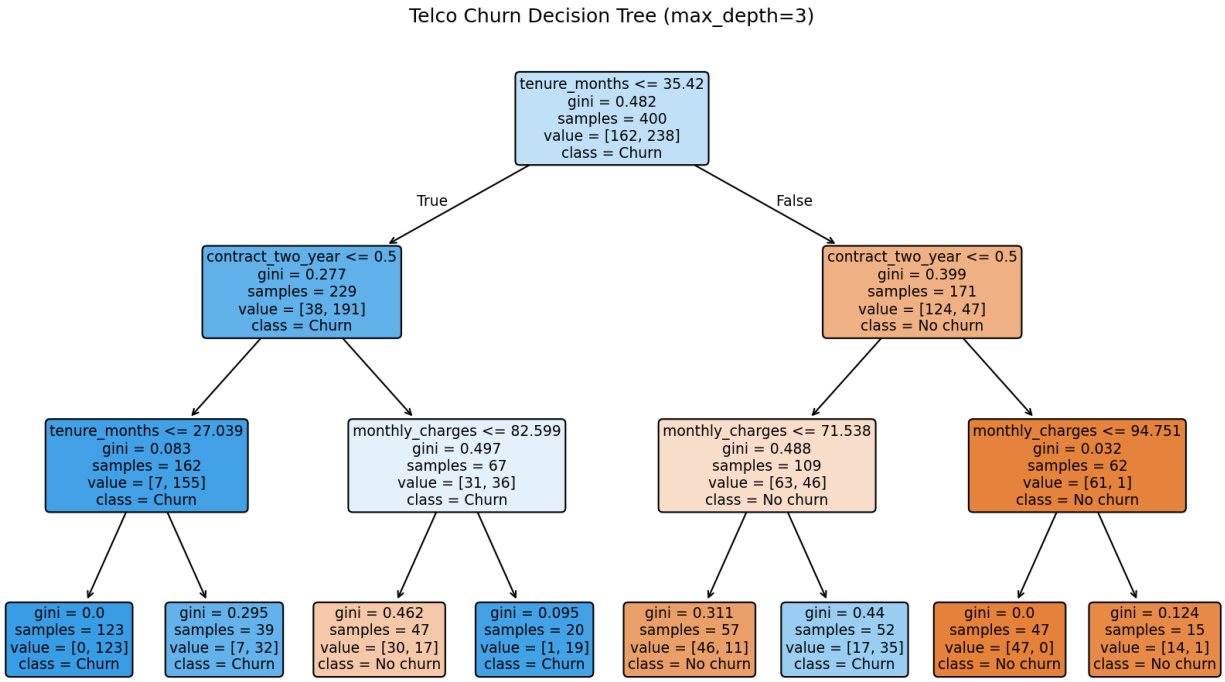
What is **permutation importance**, and when does it usually beat impurity-based importance?

a) Permutation importance shuffles a single feature's values in a held-out set and measures how much performance drops — features that were genuinely predictive cause large drops; useless features cause near-zero drops. It is more reliable than impurity-based importance for high-cardinality features and across model types
b) Permutation importance is identical to impurity-based importance
c) It is only used for clustering
d) It is faster than computing Gini
e) It always produces zero importance for every feature

☐ *Jump to solution*

Question 33

The decision tree below predicts customer churn from telecom data.



Which statement is BEST supported by the diagram?

a) The tree uses only one feature
b) The first split (root) is on `tenure_months`; the tree then routes customers to deeper splits on `monthly_charges` and `contract_two_year` before reaching leaves that predict "Churn" or "No churn." A new customer's prediction is found by walking the rules from the root down to the leaf
c) The tree predicts the same class for every customer
d) The tree has no leaves
e) The tree is a regression tree

☐ *Jump to solution*

Question 34

Why are decision trees often described as **interpretable**?

a) They are written in Python, which is interpretable
b) They make predictions extremely fast
c) A prediction can be explained as a series of yes / no rules along a single path from root to leaf — trivially translatable into the kind of "because A AND B AND not C" sentence a regulator, manager, or applicant can understand
d) They use only one feature
e) Their training accuracy is always 1.00

☐ *Jump to solution*

Question 35

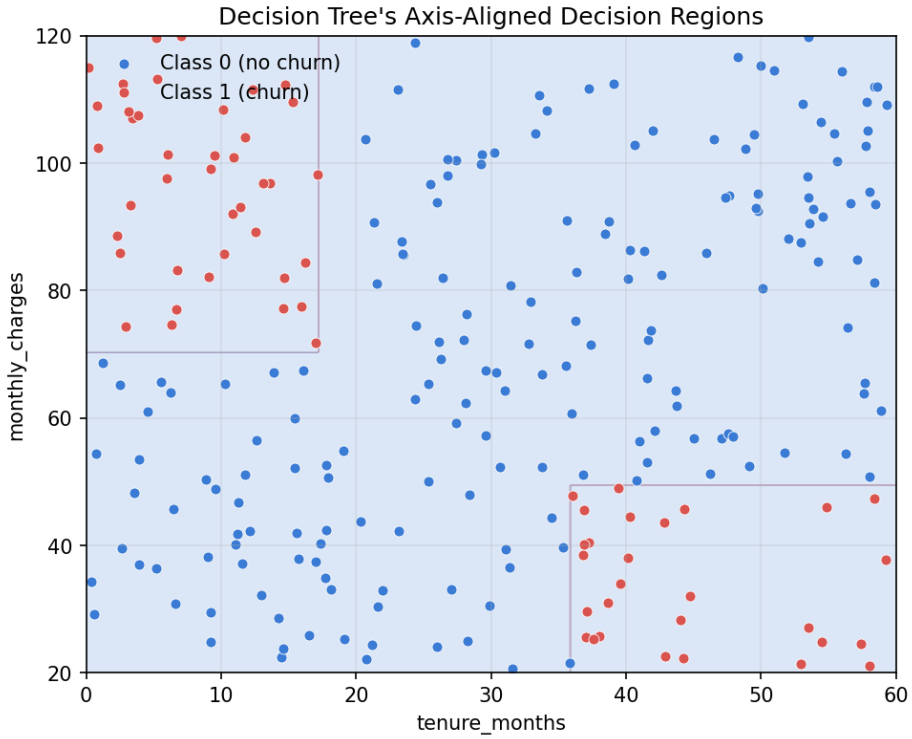
A regulator asks a bank to explain why a particular loan applicant was denied. Which model is EASIEST to defend on individual-level interpretability?

a) A single shallow decision tree with named features — the denial reduces to a short path of feature thresholds
b) A neural network's hidden-layer activations
c) A 500-tree Random Forest with hundreds of complex interactions
d) The training-loss curve of a gradient-boosting model
e) A K-Means cluster assignment

☐ *Jump to solution*

Question 36

The plot below shows the decision regions a tree carves out in a 2D feature space.



Which statement is BEST supported by the plot?

a) The tree's decision regions are smooth curves
b) The tree predicts the same class everywhere
c) The decision regions are AXIS-ALIGNED rectangles — every split is on a single feature at a threshold, so the boundaries are horizontal or vertical line segments. Trees cannot directly produce a diagonal boundary; they approximate one with a staircase of axis-aligned splits

d) The features are perfectly correlated
e) The tree uses a kernel function

☐ *Jump to solution*

Question 37

Why are single decision trees often described as UNSTABLE?

a) Because sklearn picks random splits at every node
b) Because they only work on tabular data
c) Small changes to the training data can produce very different trees — one early split that flips can cascade through the entire structure. This is exactly the instability that Random Forest's averaging is designed to remove
d) Because their predictions fluctuate during training
e) Because their training loss never converges

☐ *Jump to solution*

Question 38

A team trains a decision tree on a single train/test split and gets a great test score. They re-split (different `random_state`) and get a very different score. What does this tell them?

a) The team should report the original score and ignore the rest
b) Single trees are sensitive to the specific train/test split. Use cross-validation to get a stable estimate of generalization, and consider an ensemble (Random Forest, Gradient Boosting) for more robust predictions
c) sklearn's tree has a bug
d) The dataset is too small to train any model
e) Switch to logistic regression

☐ *Jump to solution*

Question 39

Do decision trees require feature SCALING (standardization, MinMax, etc.)?

a) Yes — sklearn refuses to train without scaling
b) Yes — trees use Euclidean distance internally
c) Yes, but only for categorical features
d) No — splits compare values WITHIN a single feature against a threshold. Monotonic transformations of a feature do not change which split is best, so scaling has no effect on tree predictions
e) Only for regression trees

☐ *Jump to solution*

Question 40

Do decision trees require categorical features to be ONE-HOT encoded in scikit-learn?

a) No — sklearn auto-detects strings and handles them
b) No — categorical features are silently dropped
c) Yes — only one-hot is allowed
d) Yes — sklearn's tree algorithm requires numeric inputs, so categorical strings must be converted (one-hot or ordinal)
e) Only if the target is binary

☐ *Jump to solution*

Question 41

A team's classification tree on a 95% / 5% imbalanced dataset has high accuracy but the rare class has near-zero recall. Which combination of fixes is MOST appropriate?

a) Increase `max_depth` to 50
b) Set `random_state=0`
c) Drop the rare class entirely
d) Replace with logistic regression
e) Set `class_weight="balanced"` (so minority errors are penalized more heavily during fitting), tune the threshold on `predict_proba` (the default 0.5 may be too high), and evaluate with precision / recall on the rare class — not just accuracy

☐ *Jump to solution*

Question 42

`tree.predict_proba(X_test)` on a binary classifier returns:

a) An array of integer class labels
b) A pandas DataFrame
c) A scalar accuracy
d) The training labels
e) An array of shape (n_test, 2) where each row gives the proportion of each class at the leaf the row landed in — essentially the leaf's class frequencies. Useful for thresholding when costs are asymmetric

☐ *Jump to solution*

Question 43

A regression tree at a leaf has training targets [40, 42, 45, 50, 55] and predicts the mean (~46.4) for any new row at that leaf. Which statement is TRUE about regression-tree predictions?

a) The tree extrapolates linearly beyond the training range
b) The tree fits a polynomial in each leaf
c) The tree fits a linear regression in each leaf
d) The tree returns a constant within each leaf — the same predicted value for every row that lands there. Predictions cannot exceed the maximum training target or fall below the minimum (no extrapolation)
e) The tree returns NaN

☐ *Jump to solution*

Question 44

Consider:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

tree = DecisionTreeClassifier(
    max_depth=4,
    min_samples_leaf=50,
    random_state=42,
)
tree.fit(X_train, y_train)
print(tree.score(X_test, y_test))
```

Which statement about this code is BEST?

a) The tree is PRE-PRUNED (`max_depth=4`, `min_samples_leaf=50`) — these reduce overfitting risk by capping complexity. It trains on `X_train` and reports test-set accuracy; reproducibility is fixed by `random_state=42`
b) The tree will crash because `max_depth` and `min_samples_leaf` cannot be used together
c) The score function returns the loss
d) `random_state=42` is illegal
e) The code trains on the test set

☐ *Jump to solution*

Question 45

Consider:

```
from sklearn.tree import DecisionTreeRegressor
reg = DecisionTreeRegressor(max_depth=6, min_samples_leaf=20, random_state=0)
reg.fit(X_train, y_train)
print(reg.score(X_test, y_test))
```

What does `reg.score(X_test, y_test)` return for a regression tree?

a) The accuracy
b) The mean squared error
c) The Gini impurity
d) The number of leaves
e) The test-set R^2 — the standard `.score()` for sklearn regressors

☐ *Jump to solution*

Question 46

Consider:

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 6))
plot_tree(tree, feature_names=feature_names,
          class_names=["No churn", "Churn"],
          filled=True, rounded=True, ax=ax)
plt.show()
```

What does `plot_tree` produce?

a) A bar chart of feature importances
b) A scatter plot of the data
c) A confusion matrix
d) A graphical visualization of the fitted tree — every internal node shows its split rule, impurity, sample count, and class distribution; leaves show the predicted class. Color-coding (`filled=True`) reflects the dominant class at each node
e) The training loss curve

☐ *Jump to solution*

Question 47

Consider:

```
import pandas as pd
imp = pd.Series(tree.feature_importances_, index=feature_names)
print(imp.sort_values(ascending=False).head(10))
```

What is this snippet doing?

a) Predicting on the test set
b) Reporting the test accuracy
c) Saving the model to disk
d) Computing Gini impurity
e) Sorting the tree's feature importances in descending order and printing the top 10 — the standard way to inspect "which features did this tree rely on most?"

☐ *Jump to solution*

Question 48

A team is asked: "How DEEP did the tree actually grow?" Which attribute of a fitted `DecisionTreeClassifier` answers this directly?

a) `tree.score(X, y)`
b) `tree.feature_importances_`
c) `tree.tree_.max_depth` — exposes the actual depth of the fitted tree (which can be less than the `max_depth` hyperparameter if the data ran out of useful splits)
d) `tree.predict(X)`
e) `tree.classes_`

☐ *Jump to solution*

Question 49

Which scenario makes a decision tree the WRONG model choice?

a) Tabular data with mixed numeric and categorical features
b) When you need transparent, individual-decision explanations for a small audience
c) When you want a fast prototype
d) When the dataset has roughly 5,000 rows
e) When the underlying relationship is genuinely linear and smooth — a single tree forces axis-aligned step boundaries and approximates the smooth surface poorly compared with linear / logistic regression

☐ *Jump to solution*

Question 50

Which scenario makes a decision tree a GOOD baseline choice?

a) When you need calibrated probabilities for risk quoting
b) When the relationship has clear non-linear interactions, mixed feature types, and you want a quickly-trained, interpretable starting model
c) When the dataset is fewer than 30 rows
d) When you need to handle 1 million-dimensional embeddings
e) When the target is missing for half the rows

☐ *Jump to solution*

Question 51

A team has 50,000 customer rows and wants the most ROBUST tree-based model — they care about test accuracy more than interpretability. Single tree, Random Forest, or Gradient Boosting first?

a) Single tree — simpler is always better
b) Random Forest or Gradient Boosting first — averaging or sequentially correcting many trees fixes the single-tree instability and typically yields much better test accuracy. A single tree is best as a baseline or when interpretability dominates
c) K-Means
d) Logistic regression
e) The single tree, but with `max_depth=1`

☐ *Jump to solution*

Question 52

A team trains a tree to predict employee attrition. Their `feature_importances_` plot has `years_at_company` as the top feature. Which statement BEST reflects what this tells them?

a) Working at the company causes attrition
b) Older employees should be let go
c) `years_at_company` is the feature the tree relied on most for its splits — that is, knowing this feature reduced impurity the most across the tree. It does not establish a causal effect of tenure on attrition; tenure may be a proxy for other factors (career stage, role match, manager changes)
d) `years_at_company` is the only feature that matters
e) Importance is invalid for HR data

☐ *Jump to solution*

Question 53

A telecom churn tree has the path `Tenure < 6 months → MonthlyCharges > $80 → No customer-service calls → P(churn) = 0.83`. Describe the customer this leaf represents.

a) A long-tenured customer who calls support frequently
b) A randomly selected customer
c) A loyal customer in the lowest price tier
d) A new subscriber on a premium plan with no support contact — likely under-engaged with the product and high-risk for churn. Targeting this segment with onboarding outreach or a promotional offer is the natural next step
e) A customer who has already churned

☐ *Jump to solution*

Question 54

A team trains a deep `DecisionTreeClassifier` on 500 noisy training rows. Train accuracy is 1.00; test accuracy is 0.55. What is the SINGLE most effective change?

a) Add 100 more features
b) Switch to `criterion="entropy"`
c) Delete the test set
d) Apply pruning — lower `max_depth`, raise `min_samples_leaf`, or set `ccp_alpha` — and pick the level via cross-validation. With 500 rows and high noise, a simpler tree typically generalizes far better
e) Set `random_state=0`

☐ *Jump to solution*

Question 55

A bank wants ONE interpretable, regulator-friendly model for credit decisions. They are willing to accept a small accuracy loss for clarity. End-to-end recipe?

a) Train a 500-tree Random Forest with default settings; report only AUC
b) Train a single `DecisionTreeClassifier` with `class_weight="balanced"`, modest `max_depth` (e.g., 4–6), `min_samples_leaf` chosen by cross-validation, plot the tree, and report the rules along the path used for each decision; include `feature_importances_` (cross-checked with permutation importance) and a confusion matrix at the chosen threshold
c) Use only KNN with k=1
d) Drop the threshold and report all probabilities
e) Train a deep neural network

☐ *Jump to solution*

Part 2: Answers and Explanations

Answer 1

Correct: (a). A decision tree partitions the feature space by asking yes/no questions about the features at each internal node, routing each row down a path to a "leaf" that produces a prediction (a class for classification trees, a numeric value for regression trees).

☐ *Back to question*

Answer 2

Correct: (c). Decision trees solve both classification (`DecisionTreeClassifier`) and regression (`DecisionTreeRegressor`). The mechanics are similar; only the leaf prediction and splitting criterion differ.

☐ *Back to question*

Answer 3

Correct: (e). A classification tree at a leaf predicts the majority class and reports the leaf's class proportion as the predicted probability. Here, 60/80 = 0.75 churn probability — and "Churn" as the predicted class.

☐ *Back to question*

Answer 4

Correct: (c) The mean. A regression tree's leaf prediction is the average of the training-target values at that leaf — here, $(300 + 320 + 330 + 340 + 360)/5 = $330K.

☐ *Back to question*

Answer 5

Correct: (e). Trees natively capture non-linearity, interactions, and step-changes through their splits. Linear / logistic regression require explicit feature engineering (polynomial terms, interactions, transforms) to do the same — so trees often outperform on messy real-world tabular data.

☐ *Back to question*

Answer 6

Correct: (d). A path is a conjunction of split conditions: "feature_a > 5 AND feature_b $\leq$ 10 AND …". This is also the natural human-readable explanation of any single prediction the tree makes.

☐ *Back to question*

Answer 7

Correct: (b). Gini impurity for a binary node is $1 - (p^2 + (1-p)^2)$. It hits its minimum (0) at a pure node ($p = 0$ or 1) and its maximum (0.5) at a 50/50 split.

☐ *Back to question*

Answer 8

Correct: (c) 0.00. A pure leaf has $p = 1$ and $1 - p = 0$, giving Gini = $1 - (1 + 0) = 0$. No chance of misclassifying a randomly drawn sample from this leaf.

☐ *Back to question*

Answer 9

Correct: (d) 0.50. $1 - (0.5^2 + 0.5^2) = 1 - 0.5 = 0.50$. This is the maximum Gini for binary classification — the worst-case mix.

☐ *Back to question*

Answer 10

Correct: (c) 0.375. $p_{\text{churn}} = 0.25, p_{\text{stay}} = 0.75$. Gini = $1 - (0.0625 + 0.5625) = 1 - 0.625 = 0.375$.

☐ *Back to question*

Answer 11

Correct: (c) 0.42. $p_{\text{churn}} = 60/200 = 0.30, p_{\text{stay}} = 0.70$. Gini = $1 - (0.09 + 0.49) = 1 - 0.58 = 0.42$.

☐ *Back to question*

Answer 12

Correct: (b) 0.5. For binary classification, Gini reaches its maximum at the 50 / 50 split. For K classes, the maximum is $1 - 1/K$.

☐ *Back to question*

Answer 13

Correct: (a). Both criteria measure node impurity and produce nearly identical trees in practice. Gini avoids the logarithm and is slightly faster — and is sklearn's default. Pick whichever is more familiar to your stakeholders.

☐ *Back to question*

Answer 14

Correct: (b). A regression tree picks splits that minimize the weighted MSE (variance) within the children. Gini and entropy do not apply to continuous targets.

☐ *Back to question*

Answer 15

Correct: (a). Information gain = parent impurity − weighted average child impurity. The tree's split-finding algorithm picks the split that maximizes this number.

☐ *Back to question*

Answer 16

Correct: (a) 0.22. Weighted child Gini = $(80/200) \times 0.40 + (120/200) \times 0.20 = 0.16 + 0.12 = 0.28$. Information gain = parent − weighted children = $0.50 - 0.28 = 0.22$. The tree would compare this to every other candidate split and pick the one with the highest information gain.

☐ *Back to question*

Answer 17

Correct: (e). Maximum information gain means the children are as pure as possible relative to the parent. Pure children mean confident leaf predictions — exactly what the tree wants to maximize.

☐ *Back to question*

Answer 18

Correct: (d). `max_depth` caps the number of levels in the tree, which directly limits its complexity. Smaller `max_depth` = simpler tree = less overfitting risk but possibly underfitting.

☐ *Back to question*

Answer 19

Correct: (a). `min_samples_leaf` requires every leaf to contain at least the specified number of training samples. Splits that would create a smaller leaf are rejected, keeping the tree from carving out tiny pockets that fit a handful of training rows.

☐ *Back to question*

Answer 20

Correct: (c). A node must have at least `min_samples_split` samples to be eligible for further splitting. Together with `min_samples_leaf` and `max_depth`, it forms the standard pre-pruning toolkit.

☐ *Back to question*

Answer 21

Correct: (b). `ccp_alpha` controls cost-complexity pruning. After fitting a full tree, sklearn prunes back branches whose contribution to predictive accuracy is smaller than the alpha penalty. Tune via the `cost_complexity_pruning_path` method.

☐ *Back to question*

Answer 22

Correct: (b). `random_state` seeds the tie-breaking RNG used when several candidate splits are equally good. Setting it gives the same tree every run — essential for reproducibility, debugging, and apples-to-apples model comparison.

☐ *Back to question*

Answer 23

Correct: (e). `max_features="sqrt"` restricts each split to a random subset of $\sqrt{p}$ features. This is one of the two key sources of randomness Random Forest uses (along with bootstrap sampling) to make individual trees diverse.

☐ *Back to question*

Answer 24

Correct: (e). `class_weight="balanced"` re-weights the loss inversely proportional to class frequency. Combined with threshold tuning and per-class evaluation, it is the standard first remedy for imbalanced classification with trees.

☐ *Back to question*

Answer 25

Correct: (d). Pre-pruning stops the tree from growing too complex. Post-pruning grows the full tree and then trims branches that do not improve out-of-sample performance. Both work; pre-pruning is faster, post-pruning is often more accurate.

☐ *Back to question*

Answer 26

Correct: (a). Without restrictions a tree carves arbitrarily small regions of feature space, eventually reaching leaves with only a few (or one) training points. Late-level splits fit training noise rather than real signal — overfitting at its most pure.

☐ *Back to question*

Answer 27

Correct: (a). A 31-point gap between train and test accuracy is the classical overfitting signature. Pre-prune by lowering `max_depth` and raising `min_samples_leaf`; post-prune via `ccp_alpha`; pick the level by cross-validation. If the gap remains large, an ensemble (Random Forest, Gradient Boosting) is the next step.

☐ *Back to question*

Answer 28

Correct: (d). The hallmark overfitting curve: train accuracy keeps rising as the tree grows, but test accuracy peaks at a moderate depth and then declines. The peak's `max_depth` is a good first choice; cross-validation sharpens it further.

☐ *Back to question*

Answer 29

Correct: (e). `feature_importances_` reports the normalized total reduction in impurity attributable to each feature, summed across all the splits in the tree. By construction the values are non-negative and sum to 1.0.

☐ *Back to question*

Answer 30

Correct: (a). Reading the bars: `tenure` (0.45) is by far the most-used split feature, followed by `MonthlyCharges` (0.22) and `Contract_Two year` (0.18). `gender_Male` (0.06) was barely used. Importance reflects how much the tree relied on each feature, not whether the feature CAUSES the outcome — a critical distinction for business stakeholders.

☐ *Back to question*

Answer 31

Correct: (b). ID columns are unique per row, so the tree can isolate each training row with a single split. This drives the impurity reduction sky-high on training data while contributing nothing on new customers. Drop ID-like columns and verify the remaining importances with permutation importance.

☐ *Back to question*

Answer 32

Correct: (a). Permutation importance measures the drop in held-out performance when a feature is shuffled. It does not reward high-cardinality features just for being unique, and it is comparable across model types. Use `sklearn.inspection.permutation_importance` as a sanity check on impurity-based importance.

☐ *Back to question*

Answer 33

Correct: (b). The diagram shows a top-down tree of yes / no questions. The root splits on tenure; subsequent splits use other features; leaves carry the predicted class. To predict for a new customer, you walk the rules from the root down to whichever leaf matches.

☐ *Back to question*

Answer 34

Correct: (c). A single tree's prediction is just a path of feature thresholds — directly readable as "because A AND B AND not C ☐ outcome." That is what makes trees one of the few models a regulator or a customer can audit at the level of an individual decision.

☐ *Back to question*

Answer 35

Correct: (a). A shallow decision tree's path is short, named, and directly maps to a written rationale ("credit score < 620 AND DTI > 0.45 ☐ denied"). Random Forests and neural networks are far harder to defend at the individual-decision level without auxiliary explanation tooling.

☐ *Back to question*

Answer 36

Correct: (c). Each tree split is a single-feature threshold, so every boundary segment is parallel to a feature axis. To approximate a diagonal, the tree builds a staircase of axis-aligned splits — easy to see in the plot. This is also why trees often need many splits to capture genuinely smooth, diagonal boundaries.

☐ *Back to question*

Answer 37

Correct: (c). A small change in training data can change which feature wins the root split — and from that point the rest of the tree differs. This instability is exactly why ensembling many trees (Random Forest, gradient boosting) tends to outperform any single tree.

☐ *Back to question*

Answer 38

Correct: (b). Single trees are sensitive to the train/test split. Cross-validation gives a more stable estimate; an ensemble (Random Forest, Gradient Boosting) is the standard fix when stability matters.

☐ *Back to question*

Answer 39

Correct: (d). Trees split on a single feature at a numeric threshold. Any monotonic transformation of that feature (multiplying by 100, taking the log) produces an equivalent split with a transformed threshold. So scaling has no effect on tree structure or predictions — unlike linear / logistic regression and distance-based models, which DO need scaling.

☐ *Back to question*

Answer 40

Correct: (d). Scikit-learn's tree algorithm operates on numeric arrays, so categorical strings must be converted (one-hot or ordinal). The tree itself does not strictly NEED one-hot encoding (a label-encoded integer can yield the same effective splits), but the encoding choice can matter for interpretability and for high-cardinality columns where one-hot explodes feature count.

☐ *Back to question*

Answer 41

Correct: (e). The standard recipe for imbalanced trees: `class_weight="balanced"` during fitting, threshold tuning on `predict_proba` for the deployment cutoff, and per-class evaluation (precision / recall / F1 on the rare class) instead of overall accuracy.

☐ *Back to question*

Answer 42

Correct: (e). `predict_proba` for a tree returns the leaf's class proportions — i.e., for each row, the proportion of each class among the training samples that landed at the same leaf. Useful for ranking and threshold tuning, though the values can be coarse (limited to the leaf's resolution).

☐ *Back to question*

Answer 43

Correct: (d). A regression tree returns a constant within each leaf (the training mean of that leaf's targets). It cannot extrapolate beyond the range of training targets. Smooth interpolations require a different model family (linear regression, splines, neural networks, gradient boosting).

☐ *Back to question*

Answer 44

Correct: (a). The tree is pre-pruned by `max_depth=4` and `min_samples_leaf=50`, both reducing overfitting risk. `train_test_split` with a seeded `random_state` makes the split reproducible. `tree.score(X_test, y_test)` returns test-set accuracy for a classifier (R^2 for a regressor).

☐ *Back to question*

Answer 45

Correct: (e). Sklearn regressors' `.score(X, y)` returns R^2 by default. For accuracy or a different metric, compute it directly via `sklearn.metrics`.

☐ *Back to question*

Answer 46

Correct: (d). `plot_tree` draws the fitted tree: each internal node shows the split, impurity, sample count, and class distribution; leaves show the predicted class. With `filled=True`, color signals the dominant class. Combined with `feature_names` and `class_names`, this is one of the most useful interpretability tools for a single tree.

☐ *Back to question*

Answer 47

Correct: (e). Wrapping `feature_importances_` in a Series with the feature names and sorting descending is the idiom for "which features did the tree rely on most?" — a one-liner that produces a clean, sorted ranking ready to discuss with stakeholders.

☐ *Back to question*

Answer 48

Correct: (c). `tree.tree_.max_depth` returns the actual maximum depth the fitted tree reached. It can be smaller than the `max_depth` hyperparameter if the data ran out of useful splits (`min_samples_split`, `min_samples_leaf`, or impurity exhaustion).

☐ *Back to question*

Answer 49

Correct: (e). A single tree forces axis-aligned step boundaries; if the underlying relationship is genuinely smooth and linear, a linear model captures it more efficiently and with better interpolation behavior. Trees shine on non-linearities and interactions, not on smooth linear surfaces.

☐ *Back to question*

Answer 50

Correct: (b). Trees are a strong baseline for tabular data with mixed feature types and non-linear interactions, and they train fast and produce a transparent first model — perfect first step before reaching for ensembles or more complex models.

☐ *Back to question*

Answer 51

Correct: (b). Single trees are unstable and rarely top the leaderboard. Random Forest and Gradient Boosting (XGBoost, LightGBM) almost always outperform a single tree on tabular data — at the cost of interpretability. Use the single tree as a baseline or when individual-decision interpretability dominates.

☐ *Back to question*

Answer 52

Correct: (c). `feature_importances_` reports model usage, not causation. A high importance for `years_at_company` may reflect a real driver, a proxy for unobserved factors, or a confound with another variable. Treat it as a hypothesis to investigate, not a causal claim.

☐ *Back to question*

Answer 53

Correct: (d). New, high-paying, low-engagement subscribers are textbook high-churn-risk profiles. A 0.83 leaf probability is a strong signal — Netflix-style retention teams target this segment with onboarding flows or a discount.

☐ *Back to question*

Answer 54

Correct: (d). With 500 noisy rows, deep trees memorize the training set. Pre-prune (`max_depth`, `min_samples_leaf`) or post-prune (`ccp_alpha`); pick the level via cross-validation. If the gap remains large, an ensemble is the next step.

☐ *Back to question*

Answer 55

Correct: (b). The standard regulator-friendly recipe: a pre-pruned single tree balanced for class imbalance, with hyperparameters chosen by cross-validation, the tree itself plotted, the rules along each decision path documented, and feature importances cross-checked with permutation importance. Add a confusion matrix at the chosen threshold and you have a defensible, transparent model card.

☐ *Back to question*